🎙️ SPEECH D'ENTRAÎNEMENT ORAL — BLOC 3 : PIPELINE STREAMING & FRAUDE (KAFKA / AIRFLOW)

Certification Architecte en Intelligence Artificielle (RNCP 41993 — Niveau 7)
Durée du Speech : 5 Minutes Chrono *(suivi de 15 min de Q&A)*
Alignement Livrables (`01_LIVRABLES`) : Synchronisé à 100 % avec `ARCHITECTURE_PIPELINE_FRAUD_BLOC3.pptx` *(7 slides)*, `RAPPORT_ARCHITECTURE_TECHNIQUE_PIPELINE.md` et `DEMO_ET_LIEN_GITHUB.md`.
Posture : Tu es l'**Architecte Data & IA** qui a conçu et déployé le pipeline hybride de détection de fraude bancaire en temps réel (**Apache Kafka KRaft + FastAPI + HistGradientBoosting**) et d'audit batch (**Apache Airflow**).

⏱️ RÉCAPITULATIF DU CHRONO (5 MINUTES — SLIDES 1 À 7)

Chrono	Slide	Titre Exact de la Diapositive (`01_LIVRABLES`)	Compétence RNCP Validée
00:00 – 00:50	**Slides 1 & 2**	*1. Enjeux Stratégiques, Problématique & Exigences Métier*	**C1** (Cadrage & Besoins)
00:50 – 02:10	**Slides 3 & 4**	*2. Schéma d'Architecture (5 Modules Jedha) & 3. Composants Techniques Détaillés*	**C2 & C3** (Architecture & Orchestration)
02:10 – 03:10	**Slide 5**	*4. Contrôle Qualité des Flux, Résilience & Dead Letter Queue (C5)*	**C5** (Qualité & Résilience)
03:10 – 04:10	**Slide 6**	*5. Sécurité, Confidentialité des Données & Conformité PCI-DSS (C4)*	**C4** (Sécurité & Privacy)
04:10 – 05:00	**Slide 7**	*6. Synthèse des Livrables & Démonstration du Pipeline en Production*	**C3** (Industrialisation)

🗣️ TEXTE À LIRE À L'ORAL (STYLE PROMPTEUR FLUIDE)

🚨 SLIDES 1 & 2 — Enjeux Stratégiques, Problématique & Exigences Métier *(⏱️ 00:00 – 00:50 | 50 sec)*

« Bonjour à tous. Pour ce Bloc 3 consacré à la conception et à la mise en œuvre de pipelines de données pour l'IA, je vous présente notre architecture de détection de fraude bancaire en temps réel.

[Passer à la Slide 2] En Union Européenne, la fraude par carte bancaire dépasse **1 milliard d'euros par an** selon la BCE. Pour protéger les paiements, nous devons répondre à une **double exigence métier** aux temporalités opposées :

1. **En Temps Réel :** notifier et bloquer **immédiatement** chaque paiement suspect en quelques millisecondes.
2. **En Batch Quotidien :** générer chaque matin à **06h00 UTC** un rapport d'audit consolidé récapitulant l'ensemble des flux des dernières 24 heures.

Le tout sous deux contraintes réglementaires strictes : **PCI-DSS** (interdiction absolue de stocker un numéro de carte PAN en clair) et **RGPD** (pseudonymisation des clients et isolation immédiate des

messages non conformes). »

- 👉 **[Passer à la Slide 3 puis Slide 4]**

🏗️ SLIDES 3 & 4 — Schéma d'Architecture (5 Modules) & Les 6 Composants Techniques *(⏱️ 00:50 – 02:10 | 1 min 20)*

« **[Slide 3 : Schéma d'Architecture]** Comme le montre notre schéma d'infrastructure sur la Slide 3 structuré autour des **5 modules fonctionnels** (*Build Algorithm, MLflow & Serving, Consume Data, Store Data, Leverage Data*), j'ai pris une décision architecturale clé : **ne pas utiliser Apache Airflow pour le temps réel**, mais combiner **Apache Kafka en mode KRaft** pour le streaming continu et **Airflow** pour l'audit batch matinal.

Pourquoi ? Parce qu'Airflow est un ordonnanceur par lots (batch) incapable de tenir un SLA sub-seconde, alors que Kafka traite le flux continu d'événements en quelques millisecondes.

[Passer à la Slide 4 : Les 6 Composants Détaillés] Sur la Slide 4, notre pipeline hybride Cloud / On-Premise s'articule en **6 composants découplés** :

1. **Ingestion Live :** un producteur Python connecté au flux continu de transactions de l'API Hugging Face, doté d'un mode *Replay* sécurisé garantissant une vélocité continue même en cas d'indisponibilité de l'API externe.
2. **Stream Broker : Apache Kafka en mode KRaft** (sans dépendance à ZooKeeper), avec le topic `raw-transactions` distribué sur **3 partitions équilibrées**.
3. **Inférence ML :** notre microservice **FastAPI conteneurisé (Docker / GCP Cloud Run en Scale-to-Zero)** qui score chaque requête HTTP en **moins de 15 millisecondes** (dont moins de **3 ms** de calcul modèle pur sur CPU).
4. **Alerting & Sécurité :** masquage PCI-DSS à la volée et décision automatique (`BLOCK_IMMEDIATE` ou `3DS`) dès que la probabilité de fraude atteint **50 %**.
5. **Dual Storage (Persistance Double) :** écriture simultanée dans notre **Data Lake Bronze en Parquet (compression Snappy)** partitionné par année et mois, et dans notre **base SQL opérationnelle (SQLite / PostgreSQL)** pour l'audit trail immuable.
6. **Batch Audit :** notre DAG **Apache Airflow** déclenché chaque matin à **06h00 UTC** qui génère le rapport exécutif en Markdown. »

- 🛡️ *Raccord 100 % Slide 3, Slide 4 et Rapport Technique : Tu expliques les 5 modules du schéma de la Slide 3, les 6 briques de la Slide 4, et tu fais le pont exact entre `< 15 ms` (latence HTTP de la Slide 4/6) et `< 3 ms` (temps de calcul CPU du Rapport Technique) !*
- 👉 **[Passer à la Slide 5]**

🎯 SLIDE 5 — Contrôle Qualité des Flux, Résilience & Dead Letter Queue (C5) *(⏱️ 02:10 – 03:10 | 1 min 00)*

« Sur la **Slide 5**, nous répondons à la **compétence C5** sur le contrôle qualité et la résilience des flux. En streaming, le risque majeur est la *"Poison Pill"* : un message JSON malformé ou corrompu qui ferait planter le consommateur et bloquerait toute la file de paiements.

J'ai donc implémenté une double barrière :

1. **Un Data Quality Gate strict avec Pydantic :** vérification du schéma et des attributs obligatoires (montant, coordonnées GPS, date/heure), calcul des métriques de complétude, et **rejet immédiat des montants négatifs** (comme `-150 €`) qui correspondent à des bugs ou tentatives d'exploit.
2. **Un mécanisme de Dead Letter Queue (DLQ) à deux niveaux :** tout message rejeté par Pydantic ne bloque **jamais** la file principale des transactions saines. Il est immédiatement routé vers le **topic Kafka dédié** `fraud-dlq` ET isolé dans la **table SQL** `dead_letter_queue` avec son payload brut et la cause exacte de l'erreur, permettant un rejeu asynchrone après correction. »

- 🛡️ *Raccord 100 % Slide 5 : Tu cites à la fois le topic Kafka `fraud-dlq` ET la table SQL `dead_letter_queue` ainsi que le rejet des montants négatifs, exactement comme écrit sur ta Slide 5 !*
- 👉 **[Passer à la Slide 6]**

🔐 SLIDE 6 — Sécurité, Confidentialité RGPD / PCI-DSS (C4) & Modèle ML *(⏱️ 03:10 – 04:10 | 1 min 00)*

« Sur la **Slide 6**, nous validons la **compétence C4** de sécurité et de *Privacy-by-Design* avant toute persistance sur disque :

1. **Masquage PCI-DSS au format 6+4 :** le numéro de carte bancaire (*PAN*) est systématiquement tronqué (`453288******9812`) : seuls les 6 premiers chiffres (code banque BIN) et les 4 derniers sont conservés. Aucun numéro complet n'est jamais stocké en base.
2. **Pseudonymisation RGPD (Articles 5 et 32) :** les noms et prénoms des porteurs sont hachés via **SHA-256** et masqués visuellement (`J*** D*** ID:A3F8`), garantissant la minimisation des données personnelles.
3. **Performance du Modèle & SLA** `< 15 ms` **:** notre classifieur `HistGradientBoostingClassifier` **(Scikit-Learn)** a été entraîné sur **555 719 transactions réelles (** `fraudTest.csv` **)** autour de 5 features clés : le montant (`amt`), l'heure (`hour`), l'âge (`age`), la population de la ville (`city_pop`) et surtout la **distance géodésique de Haversine (** `distance_km` **)** entre l'acheteur et le commerçant. Il atteint un **ROC-AUC de 0,9866** et un **Rappel de 92,4 %**, déclenchant une alerte `CRITICAL` ou `HIGH` dès que la probabilité atteint **50 %**. »

- 👉 **[Passer à la Slide 7]**

🚀 SLIDE 7 — Synthèse des Livrables, Cloud Run & Démonstration *(⏱️ 04:10 – 05:00 | 50 sec)*

« Pour conclure sur la **Slide 7**, l'ensemble des livrables est industrialisé et vérifiable :

- **Dépôt GitHub officiel (** `CERTIFICATION_AIA` **)** avec CI/CD Google Cloud Build et `.gitignore` strict.
- **Déploiement Serverless sur Google Cloud Run** du microservice FastAPI avec **Scale-to-Zero (** `min-instances: 0` **, 1 vCPU / 1 GiB)** pour une facturation nulle hors trafic.
- **Tracking MLflow complet** (ROC-AUC > 0,98, Rappel > 92 % et Model Registry).
- Et notre **démonstrateur (** `run_full_pipeline_demo.py` **)** ainsi que la **vidéo live** qui prouvent en une commande la double temporalité métier : l'alerte temps réel Kafka en moins de 15 ms, l'isolation dans la Dead Letter Queue et la génération du rapport matinal Airflow à 06h00.

Je vous remercie pour votre attention, et je suis à votre disposition pour vos questions ! »

🛡️ TOP 5 QUESTIONS RÉFLEXES DU JURY APRÈS CE SPEECH (RÉPONSES EN 25s)

1. **« Dans la commande `mlflow ui --backend-store-uri sqlite:///mlflow.db`, pourquoi utilisez-vous SQLite et où sont stockés les poids du modèle ? »**

   > *« MLflow sépare deux couches : le Backend Store (`sqlite:///mlflow.db`) qui stocke les métadonnées légères (hyperparamètres, métriques ROC-AUC = 0.9866, Rappel = 92.4 %, états du Model Registry) dans une base relationnelle, et l'Artifact Store (dossier `mlruns/` ou bucket Cloud S3/GCS) qui stocke les fichiers binaires lourds comme l'artefact `.pkl` du modèle `HistGradientBoostingClassifier`. »*

2. **« Sur votre Slide 6 vous indiquez `< 15 ms Latence`, et dans votre rapport technique `< 3 ms` / `< 4 ms`. Quelle est la différence ? »**

   > *« C'est la différence entre le temps de calcul CPU pur du modèle (`predict_proba` de `HistGradientBoostingClassifier` qui prend moins de 3 millisecondes en mémoire) et la latence totale de bout en bout de l'appel HTTP REST entre le consommateur Kafka et le conteneur Docker FastAPI (sérialisation JSON + réseau + validation Pydantic + inférence), qui s'exécute en moins de 15 millisecondes, bien en dessous du SLA bancaire de 100 ms. »*

3. **« Pourquoi avoir choisi `HistGradientBoostingClassifier` plutôt qu'un Random Forest ou un réseau de neurones profond ? »**

   > *« Inspiré de LightGBM, `HistGradientBoostingClassifier` regroupe les variables continues en histogrammes de 256 bins : sur les 555 719 lignes de `fraudTest.csv`, il s'entraîne 10 fois plus vite qu'un Random Forest, gère nativement le fort déséquilibre de classes (0,5 % de fraudes) et s'exécute en moins de 3 ms sur un simple vCPU sans nécessiter de carte GPU coûteuse. »*

4. **« Pourquoi stocker les transactions scorées à la fois dans le Data Lake Parquet (`data_lake/bronze/`) ET dans la base SQL opérationnelle ? »**

   > *« Parce qu'ils servent deux usages complémentaires : la base SQL (orientée lignes) permet des requêtes transactionnelles rapides pour le suivi des alertes temps réel, l'isolation de la table `dead_letter_queue` et la réconciliation du matin ; tandis que le Data Lake Parquet Snappy (orienté colonnes, compressé à 75 % et partitionné par `year/month`) est optimisé pour le* Column Pruning *lors du réentraînement batch du modèle ML sur des millions de lignes. »*

5. **« Comment garantissez-vous qu'aucune transaction n'est perdue si le consommateur Kafka redémarre ? »**

   > *« Grâce à la gestion des Offsets Kafka dans le groupe de consommateurs et aux 3 partitions du topic `raw-transactions` : l'offset n'est commité qu'après le scoring et la persistance sur disque (ou le routage vers `fraud-dlq`). Si un conteneur Consumer tombe, Kafka réassigne la partition et reprend exactement au dernier offset non validé, garantissant une sémantique* At-Least-Once *complétée par le dédoublonnage sur `trans_num`. »*